EDUGENIE

AI-Powered Educational Assistant

MILESTONE 2

Core Functionalities Development

Activity 2.1 — Module Implementation

Project Overview

EduGenie is an AI-based educational assistant designed to make learning simpler, more accessible and more interactive. The system combines generative AI with a student-friendly interface to support explanations, question answering and quiz generation. The main goal is to convert difficult academic content into clear, readable and useful learning material.

Main Objectives

- Provide simple, context-aware explanations for difficult concepts.
- Answer general knowledge and academic questions using an AI model.
- Generate multiple-choice quizzes from a supplied passage or topic.
- Return structured outputs that can be integrated into an educational application.
- Support beginners and school/college learners with an accessible study companion.

Project	EduGenie — AI-Powered Educational Assistant
Milestone	Milestone 2: Core Functionalities Development
Activity	Activity 2.1: Module Implementation
Core Modules	Explanation Module | Q&A Module | Quiz Module
AI Technologies	LaMini-Flan-T5 and Gemini-based generation

Scope: This report covers the three core functional modules, their processing logic, sample implementation approach, expected outputs, testing considerations and educational benefits.

1. System Architecture and Module Overview

EduGenie follows a modular architecture. A learner provides a topic, question or reading passage through the application interface. The request is routed to the appropriate AI module, processed by the selected model, and returned as a learner-friendly response. Separating the functions into modules makes the system easier to maintain, test and extend.

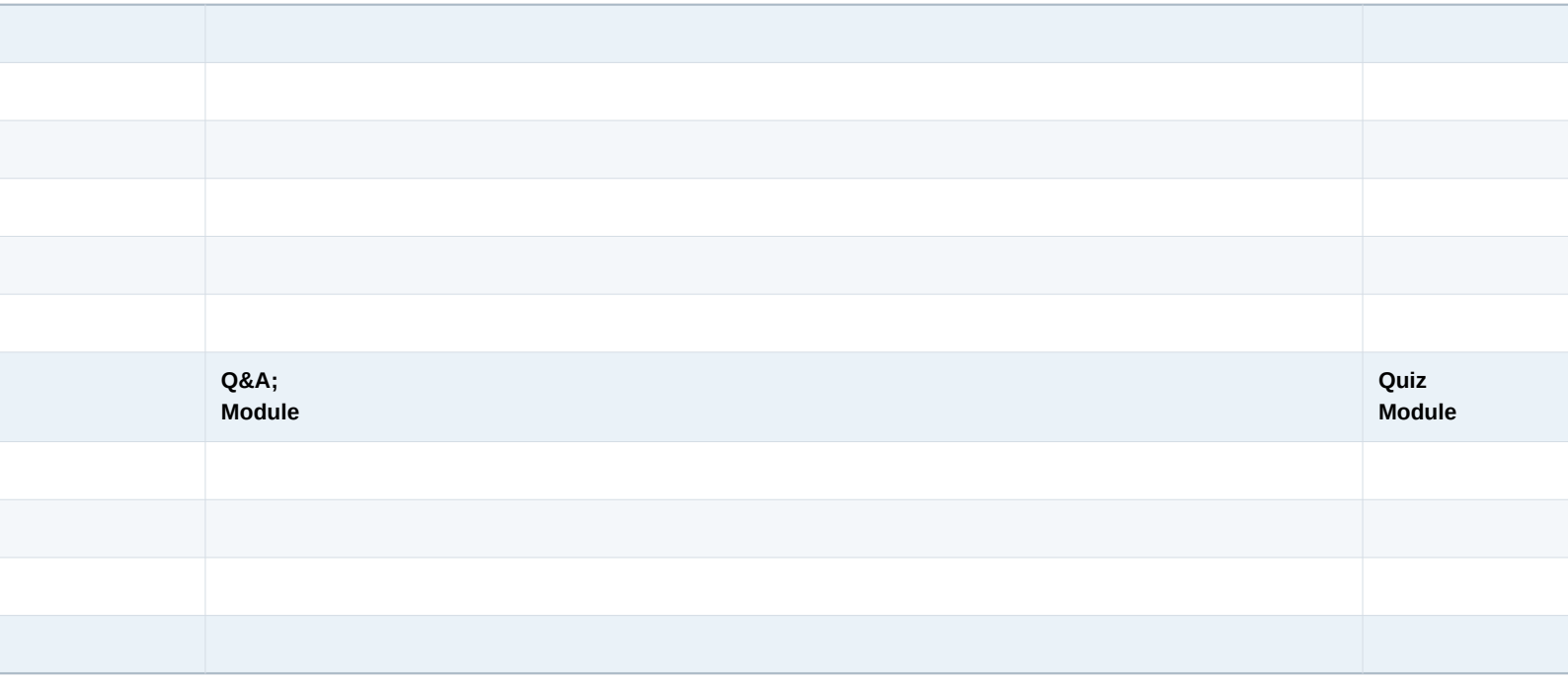

Fig. 1 — High-level EduGenie functional architecture

Module Responsibilities

Module	Input	Processing	Output
Explanation	Topic / concept	Prompt + LaMini-Flan-T5 generation	Simple explanation
Q&A	User question	Gemini-based generation	Contextual answer
Quiz	Passage / topic	Question + distractor generation	MCQs in structured format

Functional Flow

1. The learner selects a function and provides the required input.
2. The application validates and prepares the input as a model prompt.
3. The selected AI model generates the requested educational content.
4. The response is decoded, cleaned and formatted for the user.
5. The final result is displayed in a simple and readable form.

2. Explanation Module

The Explanation Module is designed for learners who need a difficult topic explained in simple language. The implementation uses the LaMini-Flan-T5 model, a lightweight text-to-text generation model. The prompt instructs the model to explain the topic clearly for a school-level learner. This encourages concise, readable and context-aware educational responses.

Working Principle

- Receive a topic such as “photosynthesis”, “Ohm’s law” or “database normalization”.
- Create a clear instruction asking the model to explain the topic simply.
- Tokenize the instruction before sending it to the model.
- Generate the response using controlled generation parameters.
- Decode the generated tokens and return the explanation as text.

Sample Implementation Logic

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

model_name = "MBZUAI/LaMini-Flan-T5-783M"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

def explain_topic(topic):
    prompt = f"Explain the concept of {topic} in a simple and clear way."
    inputs = tokenizer(prompt, return_tensors="pt")
    outputs = model.generate(
        **inputs, max_new_tokens=150,
        temperature=0.7, top_k=50, top_p=0.95
    )
    return tokenizer.decode(outputs[0], skip_special_tokens=True)
```

Fig. 2 — Simplified explanation module implementation

Generation Controls

Parameter	Purpose
max_new_tokens	Limits the maximum length of the generated explanation.
temperature	Controls variation in generated text; moderate values support readable output.
top_k	Limits token selection to a set of high-probability candidates.
top_p	Controls nucleus sampling to balance diversity and relevance.

Expected Benefits

The module reduces the difficulty of understanding unfamiliar concepts by presenting them in simpler language. It is particularly useful for beginners who may not have strong prior technical knowledge. The modular design also allows the explanation model to be replaced or upgraded without changing the overall application structure.

3. Q&A; Module

The Q&A; Module allows learners to ask direct questions and receive contextual answers. The project specification uses Gemini 1.5 Pro for this function. The application sends the question to the generative model and returns the generated response after basic error handling. This module is suitable for general knowledge, academic questions and follow-up learning queries.

```
def answer_question_with_gemini(question: str) -> str:
    try:
        model = genai.GenerativeModel("gemini-1.5-pro")
        response = model.generate_content(question)
        return response.text.strip()
    except Exception as e:
        return f"Error in Q&A: {e}"
```

Fig. 3 — Q&A; module logic

4. Quiz Module

The Quiz Module converts a given passage or topic into multiple-choice questions (MCQs). The intended implementation generates three questions, with four options for each question. The AI model is instructed to understand the passage, create relevant questions and produce plausible distractors. The final response can be represented in JSON so that the application can easily display the questions in a quiz interface.

Quiz Generation Flow

Step	Operation
1	Receive a passage or learning topic.
2	Send a structured quiz-generation prompt to the AI model.
3	Generate three relevant MCQs.
4	Create four options per question, including plausible distractors.
5	Return the result in a consistent JSON structure.
6	Display the quiz and collect learner responses.

Example JSON Structure

```
{
  "questions": [
    {
      "question": "What is the main idea of the passage?",
      "options": ["A", "B", "C", "D"],
      "answer": "B"
    }
  ]
}
```

Fig. 4 — Example structured quiz output

Integration advantage: JSON output separates the AI response from the user interface. This makes it easier to render questions on a web or mobile screen and later add scoring, feedback and progress tracking.

5. Testing, Benefits and Conclusion

Testing is required to ensure that each module accepts valid input, produces relevant output and handles unexpected conditions safely. Since generative AI can produce variable responses, evaluation should focus on relevance, clarity, factual usefulness, structure and consistency.

Module-wise Test Cases

Module	Test Input	Expected Result
Explanation	Topic: Photosynthesis	Short, clear explanation suitable for a beginner.
Explanation	Empty topic	Validation message or safe handling without crashing.
Q&A	“What is a database?”	Relevant contextual answer from the AI model.
Q&A	Network/API failure	Readable error message; application remains stable.
Quiz	One educational paragraph	Three MCQs with four options each in the expected structure.
Quiz	Very short/invalid passage	Validation or meaningful fallback response.

Key Benefits

- Simple learning: complex topics can be presented in beginner-friendly language.
- Interactive support: learners can ask questions instead of depending only on static notes.
- Active learning: generated quizzes help learners check their understanding.
- Modular development: each core feature can be tested and improved independently.
- Extensibility: future versions can add summarization, flashcards, progress tracking and personalized learning.

Limitations and Future Improvements

AI-generated answers may occasionally be incomplete or inaccurate, so important academic information should be checked against trusted learning resources. Future versions can add stronger validation, source-aware answers, response caching, user profiles, quiz scoring, learning analytics and multilingual support. Model selection can also be optimized according to response quality, speed and available resources.

Conclusion

EduGenie’s Milestone 2 establishes the core educational functionality through three complementary modules: Explanation, Q&A; and Quiz. The Explanation Module focuses on simplified learning content, the Q&A; Module supports direct academic interaction, and the Quiz Module promotes active recall through structured MCQs. Together, these components form a practical foundation for an AI-powered educational assistant that can be expanded into a complete student learning platform.

> **Final Outcome:** A modular AI learning assistant capable of explaining concepts, answering questions and generating quizzes in a student-friendly format.